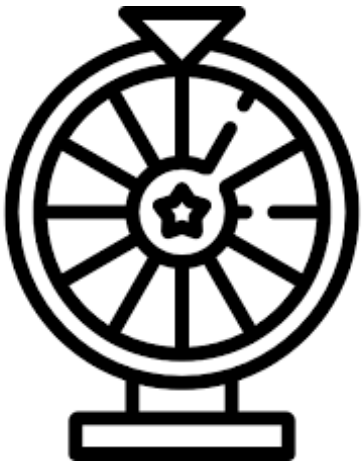
departamento
de engenharia eletrónica
e telecomunicações
e de computadores

Licenciatura em Engenharia Informática e de Computadores



JOGO DA ROLETA
(*Roulette Game*)

Projeto
de
Laboratório de Informática e Computadores
2024 / 2025 verão

Margarita Sysoeva 51403, Pilar Gonçalves 51806

13 de junho de 2025

Conteúdo

1 Descrição **1**

2 Arquitetura do Sistema **3**
2.1 *Keyboard Reader* 3
2.2 *Coin Acceptor* 3
2.3 *Serial LCD Controller* 4
2.4 *Serial Roulette Controller* 4
2.5 *Control* 4

3 Hardware **6**

4 Software **7**
4.1 Classe File Access 7
4.2 Coin Deposit 7
4.3 Statistics Manager 8
4.4 Coin Acceptor 9
4.5 TUI 9
4.6 M 10
4.7 APP 11

5 Interligações entre Hardware e Software **12**

6 Resultados Experimentais **14**

7 Conclusões **15**

8 Trabalho Futuro **15**

Referências **17**

A Roulette Game - Hardware **18**
A.1 Roulette Game.vhd 18
A.1.1 Register1.vhd 22
A.2 Atribuição de Pinos 23

B Roulette Game - Software (*Control*) **26**
B.1 Roulette Game - APP 26
B.2 M 30
B.3 TUI 34
B.4 Coin Acceptor 37
B.5 Coin Deposit 38
B.6 Statistics Manager 39
B.7 File Access 40

C Roulette Game - Simulation **42**

Lista de Figuras

1 Diagrama de blocos do jogo da Roleta (*Roulette Game*) 1
2 Arquitetura do sistema que implementa o jogo da Roleta (*Roulette Game*) . . . 3
3 Bloco *Coin Acceptor* 4
  a Diagrama de blocos 4
  b Diagrama temporal 4
4 Diagrama de blocos do módulo *Serial LCD Controller* (*SLCDC*) 4
5 Diagrama de blocos do módulo *Serial Roulette Controller* (*SRC*) 4
6 Diagrama lógico do Jogo da Roleta (*Roulette Game*) 5
7 Sinais de entrada e saída do *UsbPort* 12
8 Netlist Viewer do *Roulette Game* 22

Lista de Tabelas

1 Atribuição de pinos para o Roulette Game [4] 23

Lista de Algoritmos

1 Roulette Game Application 26
2 Maintenance 30
3 TUI 34
4 Coin Acceptor 37
5 Coin Deposit 38
6 Statistics Manager 39
7 File Access 40

1 Descrição

Este projeto implementa o jogo da Roleta (*Roulette Game*), no qual a roleta compreende números entre 0 e 9, e alfabetos entre A e D, um jogador realiza apostas premindo as teclas de um teclado correspondentes aos números/alfabetos em que pretende apostar. Por cada aposta é debitado um crédito ao saldo acumulado do jogador, podendo o jogador apostar mais do que um crédito num mesmo número. Os créditos são obtidos pela introdução de moedas no moedeiro, este só aceita dois tipos de moeda, que correspondem a dois (2) e quatro (4) créditos.
O sistema que implementa o jogo é constituído por: um PC (*Control*); um teclado de 16 teclas; um moedeiro (*Coin Acceptor*); um mostrador *Liquid Cristal Display* (*LCD*) de duas linhas com 16 caracteres; um mostrador da roleta (*Roulette Display*) e uma chave de manutenção designada por *M*, para colocação do sistema em modo de Manutenção. Na Figura 1 apresenta-se o diagrama de blocos do jogo da Roleta.

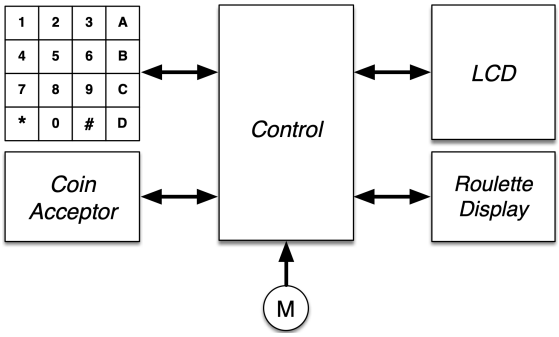


Figura 1: Diagrama de blocos do jogo da Roleta (*Roulette Game*)

Sobre o sistema proposto podem realizar-se as seguintes ações em modo de Jogo:

- **Jogo** – Inicia-se o jogo premindo a tecla '∗', e existam créditos disponíveis. Utilizando as teclas numéricas (0-9) e alfabéticas (A-D) realizam-se as apostas, retirando-se um crédito ao saldo do jogador por cada aposta realizada. O jogador termina as apostas premindo a tecla '#', o que dá início ao sorteio. Durante um tempo aleatório, o sistema simula o girar da Roleta no *Roulette Display*, permitindo ainda realizar apostas até 5 segundos antes desta parar. Ao parar a Roleta, o número sorteado e os créditos obtidos na jogada são apresentados no *Roulette Display*. Os créditos obtidos são acumulados após 5 segundos ao saldo do jogador, também apresentados no *LCD*.

No modo Manutenção podem realizar-se as seguintes ações sobre o sistema:

- **Teste** – Premindo a tecla '∗' inicia-se um jogo sem créditos e sem contabilizar os números sorteados.
- **Consultar os contadores de moedas e jogos** – Carregando na tecla 'A' permite-se a listagem dos contadores de moedas e jogos realizados.
- **Iniciar os contadores de moedas e jogos** – Premindo a tecla 'A' e em seguida a tecla '∗', o sistema de gestão coloca os contadores de moedas e jogos a zero, iniciando um novo ciclo de contagem.

- **Consultar a lista de números sorteados** – Carregando na tecla 'C' permite-se a listagem dos números sorteados.
- **Iniciar a lista de números sorteados** – Premindo a tecla 'C' e em seguida a tecla '∗', o sistema inicia um novo ciclo de estatística de números sorteados.
- **Desligar** – Premindo a tecla 'D' permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores e da Lista de Números Sorteados, são armazenadas de forma persistente em dois ficheiros de texto, por linha e com os campos de dados separados por ";". O primeiro ficheiro deverá conter o número de jogos realizados e o número de moedas guardadas no cofre do moedeiro. O segundo ficheiro deverá conter a Lista de Números Sorteados, que contém o número de saídas e os prémios atribuídos por cada número. Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

2 Arquitetura do Sistema

O sistema é implementado numa solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Serial LCD Controller* (*SLCDC*); *iii*) um módulo de interface com o mostrador da roleta (*Roulette Display*), designado por *Serial Roulette Controller* (*SRC*); *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*. Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será simulado utilizando um interruptor e um *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

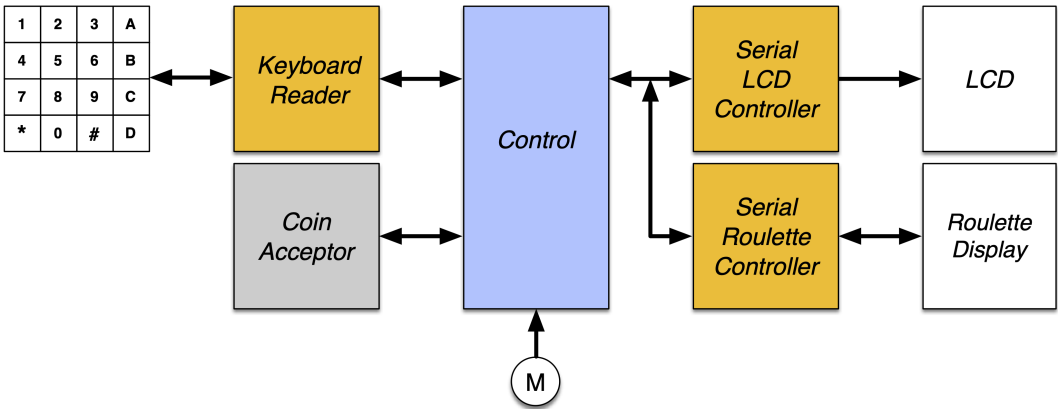


Figura 2: Arquitetura do sistema que implementa o jogo da Roleta (*Roulette Game*)

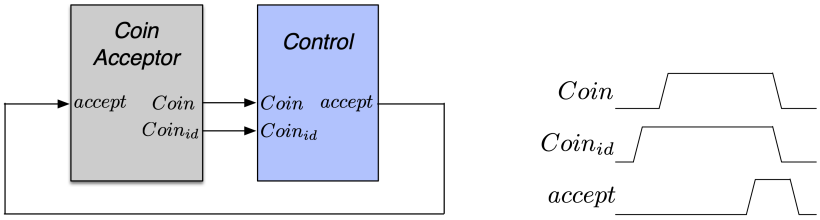
O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *SLCDC*. O *Roulette Display* é atuado pelo módulo *Control*, através do módulo *SRC*. Ppor forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *SLCDC* e *SRC* é realizada através de um protocolo série.

2.1 *Keyboard Reader*

O módulo *Keyboard Reader*, descrito em [1] é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos.

2.2 *Coin Acceptor*

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, identificando o tipo de moeda pelo sinal $Coin_{id}$. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 3b.



(a) Diagrama de blocos **(b)** Diagrama temporal

Figura 3: Bloco *Coin Acceptor*

2.3 *Serial LCD Controller*

O módulo *Serial LCD Controller* (*SLCDC*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 4, e descrito em [2].

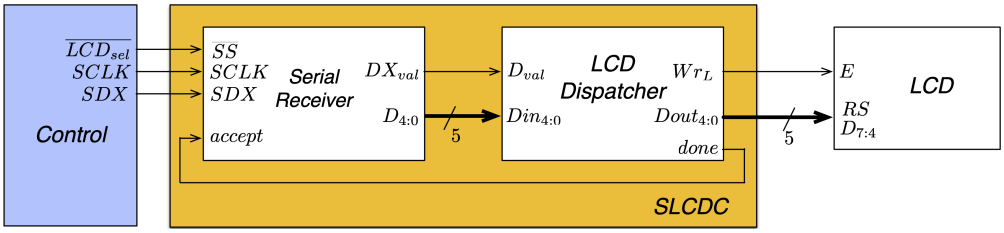


Figura 4: Diagrama de blocos do módulo *Serial LCD Controller* (*SLCDC*)

2.4 *Serial Roulette Controller*

O módulo *Serial Roulette Controller* (*SRC*) implementa a interface com o mostrador da roleta (*Roulette Display*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mostrador de pontuação, conforme representado na Figura 5, e descrito em [3].

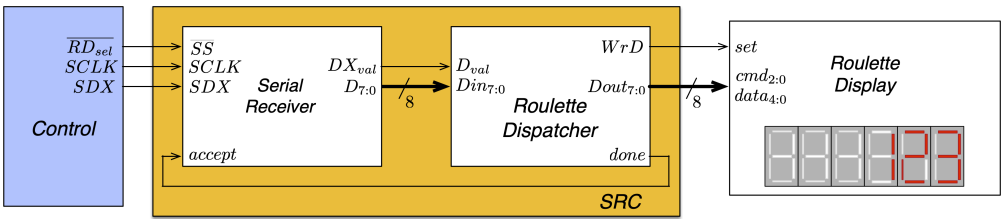


Figura 5: Diagrama de blocos do módulo *Serial Roulette Controller* (*SRC*)

2.5 *Control*

A implementação do módulo *Control* é realizada em software, usando a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 6.

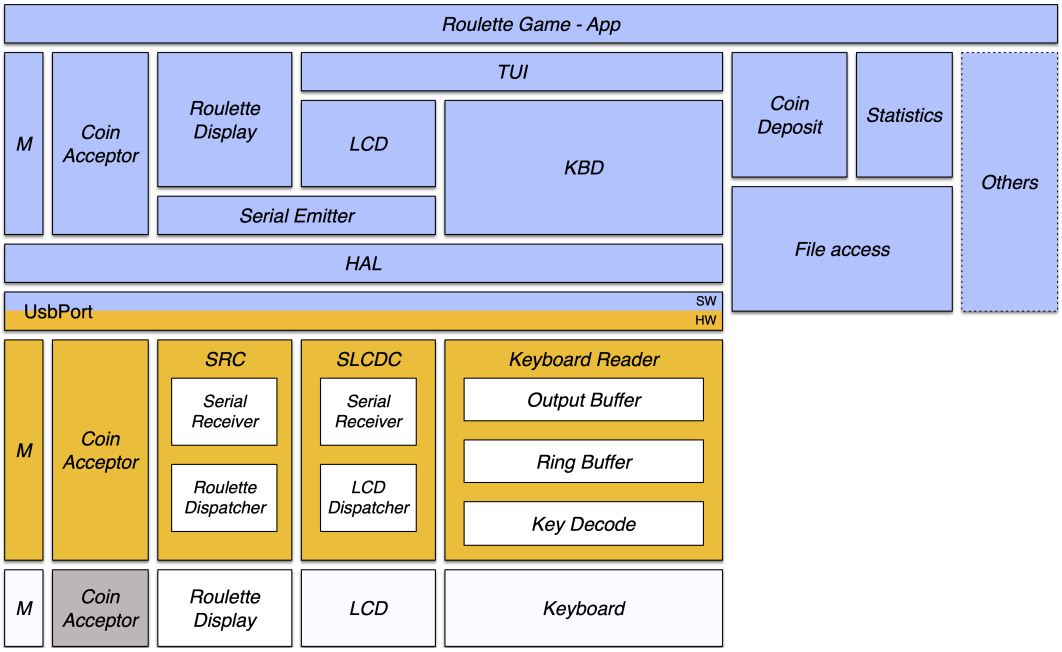


Figura 6: Diagrama lógico do Jogo da Roleta (*Roulette Game*)

3 Hardware

A entidade **Roulette_Game** representa o módulo de topo da arquitetura hardware do sistema. Este módulo estrutura e interliga os principais blocos funcionais responsáveis pela comunicação entre o PC (via UsbPort) e os periféricos físicos: o teclado matricial, o LCD (via SLCDC), o mostrador da roleta (via SRC + rouletteDisplay) e o moedeiro simulado.

Entradas e Saídas
A entidade possui as seguintes interfaces:
Entradas:
RESET, CLK: sinais globais de reset e clock.
KeyPadL, Coin, Coinid, M: sinais físicos provenientes do teclado, moedeiro e chave de manutenção.
Saídas:
KeyPadC: linhas de controlo da matriz do teclado.
Dval, D: sinal e dados da tecla detetada.
RS, E, DLCD: sinais de controlo para o LCD.
HEX0 a HEX5: saídas para os dígitos do RouletteDisplay.
accept: sinal de aceitação de moeda.

Arquitetura Estrutural
O módulo é organizado como um bloco estrutural, instanciando os seguintes componentes:
U0 – KeyboardReader: interpreta o teclado matricial, identificando a tecla premida e disponibilizando o respetivo código binário no barramento D com validação por Dval. Está ligado ao UsbPort para envio da tecla para o PC e recebe sinal de ACK como controlo de leitura.
U1 – UsbPort: interface de comunicação com o PC. Recebe dados como D, Dval, Coin, Coinid, M e envia sinais de resposta como ACK, accept e sinais de controlo série (SS, SCLK, SDX) para os módulos SLCDC e SRC.
U2 – SLCDC: módulo receptor série que interpreta comandos enviados do PC para o LCD. Converte os dados recebidos para os sinais paralelos RS, E e DLCD, usados diretamente no controlador LCD físico.
U3 – rouletteDisplay: traduz comandos e dados paralelos para os seis dígitos de 7 segmentos (HEX0–HEX5). Este bloco mostra valores como o saldo, o símbolo sorteado ou os prémios obtidos.
U4 – SRC: módulo receptor série destinado ao rouletteDisplay. Recebe comandos e dados em série (via SDX, SCLK, SS), e emite sinais paralelos para o bloco rouletteDisplay (via cmd, data, WrD).
U5 – Register1: regista os sinais série enviados do UsbPort e sincroniza-os com o clock do sistema (CLK). Isto garante a estabilidade temporal dos sinais reg_SS, reg_SCLK, reg_SDX, e reg_SS_SRC, usados pelos módulos SLCDC e SRC.

Função Global Este módulo funciona como ponte entre o hardware e o software do PC, traduzindo os eventos físicos (teclas, moedas, chave M) em sinais digitais para o sistema central, e convertendo os comandos emitidos pelo software do PC em comandos físicos para o LCD e o mostrador da roleta.

A arquitetura modular e totalmente síncrona (com base em CLK) facilita a depuração e a reutilização dos blocos. Ao dividir os blocos LCD e Roleta em dois caminhos distintos (SLCDC e SRC), permite-se comunicação paralela eficiente entre o PC e os dois periféricos visuais.

4 Software

4.1 Classe File Access

Para garantir uma persistência eficaz dos dados e estatísticas do jogo da Roleta, desenvolveu-se a classe **File Access** em *Kotlin* (descrita em B.7), responsável pela leitura e escrita dos ficheiros relacionados com estatísticas gerais e específicas do jogo. Esta classe assegura que os dados, como o número de jogos realizados, moedas armazenadas, e estatísticas individuais dos números sorteados, sejam gravados de forma estruturada e recuperáveis em sessões futuras.

A classe é composta por duas estruturas principais:
Statistics: Guarda o número total de jogos realizados e o total de moedas armazenadas.
RouletteNumberStat: Mantém registo do número de vezes que cada número foi sorteado e os respetivos prémios atribuídos.

Para a gestão eficiente destas estruturas, o módulo implementa as seguintes operações:
readStatistics() e writeStatistics(): Estas funções são responsáveis pela leitura e atualização dos dados gerais do jogo, armazenados no ficheiro statistics.txt. A informação é organizada com um valor por linha (jogos realizados e moedas armazenadas).
readRouletteStats() e writeRouletteStats(): Estas funções gerem o ficheiro Roulette_Stats.txt, que armazena as estatísticas detalhadas por símbolo sorteado. Cada linha neste ficheiro representa um símbolo da roleta, o número de vezes que foi sorteado e o total de prémios acumulados, separados por um delimitador (;).

O funcionamento da função inclui uma inicialização robusta, que verifica a existência dos ficheiros necessários. Caso não existam, ou estejam vazios, inicializa automaticamente com valores padrão, garantindo a integridade e consistência dos dados desde o arranque.

Durante a execução do jogo, a função atualiza dinamicamente estas estatísticas após cada partida ou inserção de moedas, permitindo uma monitorização em tempo real e a persistência adequada para futuras análises ou sessões de jogo.

A utilização destas funções é exemplificada claramente na função **main()**, onde se demonstra a leitura inicial dos ficheiros, a atualização das estatísticas gerais e específicas da roleta, e a escrita das alterações novamente nos ficheiros.
Esta função reforça a fiabilidade do sistema, facilitando a manutenção e garantindo a continuidade dos dados essenciais entre as sessões de jogo.

4.2 Coin Deposit

A classe **Coin Deposit**, desenvolvida em *Kotlin* (descrita em B.5), tem como principal objetivo gerir a aceitação e contabilização de moedas inseridas no sistema durante o jogo. Atua como ponte

entre o hardware de deteção de moedas (CoinAcceptor) e o sistema de estatísticas persistentes (Statistics), assegurando que cada inserção de moeda tem impacto imediato nos dados gravados.

Ao iniciar o programa, a função **initFromFile()** é utilizada para carregar os dados previamente gravados no ficheiro statistics.txt, restaurando o número de jogos realizados e o total de moedas guardadas até ao momento. Este mecanismo garante continuidade entre sessões.

Durante o funcionamento, a função **checkAndAccept()** verifica periodicamente se foi introduzida uma moeda. Caso seja detetada uma moeda válida, o sistema regista o seu valor com **getCoinValue()**, aceita-a com **acceptCoin()** e atualiza a contagem total de moedas armazenadas (coinsStored). O valor aceite é também retornado para que a lógica principal do jogo possa utilizá-lo conforme necessário.

A função **registerGame(winnings)** é responsável por atualizar as estatísticas após cada partida. Incrementa o número de jogos realizados (gamesPlayed) e subtrai do total de moedas armazenadas os créditos atribuídos ao jogador como prémio.

Por fim, **saveToFile()** grava os dados atualizados de volta no ficheiro statistics.txt, assegurando que todas as alterações ficam registadas de forma persistente.

O funcionamento completo do módulo é demonstrado na função **main()**, onde um ciclo contínuo aguarda inserções de moedas, processa-as assim que são detetadas, atualiza o ecrã com mensagens informativas, e grava os dados atualizados, garantindo uma experiência fluida e coerente.

Esta classe desempenha um papel fundamental na credibilidade e realismo do jogo da Roleta, assegurando que cada moeda tem impacto direto no progresso do jogador e que esse progresso é devidamente salvaguardado entre sessões.

4.3 Statistics Manager

A classe **Statistics Manager** (descrita em B.6) foi criada para facilitar a atualização das estatísticas detalhadas de cada número da roleta, garantindo que todos os resultados das jogadas são registados de forma simples e consistente. A ideia é manter o código principal do jogo mais limpo e legível, concentrando neste módulo a lógica de atualização das estatísticas relacionadas com os símbolos sorteados.

A função **initFromFile()** carrega os dados previamente guardados no ficheiro Roulette_Stats.txt e armazena-os numa lista mutável em memória. Esta lista contém objetos do tipo RouletteNumberStat, que indicam quantas vezes cada número já foi sorteado e os prémios totais atribuídos.

Sempre que ocorre uma jogada, a função **registerOutcome()** é chamado com o símbolo sorteado (por exemplo, 'A', '3', etc.) e o valor total ganho nessa jogada. Este método converte o símbolo para o índice correspondente na lista e atualiza os campos count e prizes desse número. Caso o símbolo não seja reconhecido, a função simplesmente ignora o pedido, evitando falhas no programa.

saveToFile() permite guardar os dados atualizados de volta no ficheiro Roulette_Stats.txt, garantindo que os registos se mantêm entre sessões.

O exemplo na **main()** mostra como este módulo pode ser usado para registar manualmente dois resultados e guardar as alterações, mostrando como é simples e direto interagir com estas estatísticas. Isto facilita muito a manutenção dos dados da roleta, tornando o jogo mais transparente e informativo para o utilizador, além de garantir a persistência dos resultados ao longo do tempo.

4.4 Coin Acceptor

A classe **Coin Acceptor** (descrita em B.4) é responsável por detetar e aceitar moedas introduzidas no sistema, atribuindo créditos ao jogador consoante o valor da moeda. O sistema suporta dois tipos de moeda: uma que vale 2 créditos e outra que vale 4 créditos. A identificação do valor da moeda é feita através do sinal **Coinid**, enquanto a deteção da sua inserção é feita com o sinal **Coin**.

O funcionamento baseia-se em três sinais definidos por máscaras:
COIN_MASK (0x40): indica se foi introduzida uma moeda.
COINID_MASK (0x20): define o tipo de moeda detetada.
ACCEPT_MASK (0x40): utilizado para sinalizar que a moeda foi aceite.

Na inicialização (**init()**), o módulo limpa o sinal de aceitação e garante que não há nenhuma moeda pendente, definindo lastCoinState como falso.

A função **hasCoin()** implementa um mecanismo de deteção por borda de subida: verifica se o estado atual do sinal COIN_MASK mudou de 0 para 1. Isto garante que a mesma moeda não é contada várias vezes seguidas.

getCoinValue() retorna o número de créditos da moeda, consoante o estado do sinal COINID_MASK: se estiver ativo (1), a moeda vale 4 créditos; caso contrário, vale 2 créditos.

Por fim, a função **acceptCoin()** simula a aceitação da moeda. Para isso, ativa o sinal ACCEPT_MASK durante 200 milissegundos e depois desativa-o, respeitando o diagrama temporal previsto no enunciado do projeto. Este comportamento permite sinalizar corretamente ao sistema que a moeda foi processada.

Na **main()** é iniciado um ciclo infinito. Neste ciclo, o sistema verifica continuamente se foi introduzida uma moeda. Quando isso acontece, obtém o seu valor e invoca acceptCoin() para a aceitar. Esta função pode ser utilizada para testar o funcionamento isolado do módulo, verificando se os sinais são corretamente interpretados e acionados.

4.5 TUI

A classe **TUI** (descrita em B.3) fornece uma interface de texto para interação com o utilizador através do LCD e do teclado matricial. Este módulo encapsula um conjunto de funcionalidades utilitárias que simplificam a escrita de mensagens no LCD, a leitura de entradas do utilizador e a exibição de animações visuais.

A função principal **tui()** inicia um ciclo infinito onde o sistema aguarda uma tecla durante 5 segundos. Quando uma tecla é premida, o seu comportamento depende do caractere introduzido:
- A tecla # limpa o ecrã e reposiciona o cursor no início.

- A tecla * alterna entre as duas linhas do LCD.
Qualquer outro caractere é escrito na posição atual, e o cursor avança automaticamente. Quando atinge o final da linha, avança para a seguinte, com rotação entre as duas linhas disponíveis.

Para além desta função de teste, o módulo fornece várias funções auxiliares:
writeCentered(line, text): escreve texto centrado na linha especificada.
showMessage(line, text, duration): mostra uma mensagem centrada durante um determinado período.
showLoading(...): exibe uma animação de carregamento com pontos progressivos.
scrollText(...): apresenta texto deslizante quando este excede os 16 caracteres do LCD.
blinkMessage(...): pisca uma mensagem na linha indicada.
readInput(prompt, maxLen): lê uma sequência de caracteres introduzidos pelo utilizador, com comprimento limitado.
write(...), clear() e waitKey(...): utilitários para escrita direta, limpeza do ecrã e espera por teclas.

Na função **main()**, as classes HAL, KBD, LCD e SerialEmitter são inicializadas. Segue-se um ciclo que apresenta um menu de opções ao utilizador. Dependendo da tecla pressionada, o sistema invoca diferentes funções do TUI, permitindo testar todas as funcionalidades de apresentação e interação. Esta main serve como ambiente de teste completo e visual para o módulo TUI, facilitando a validação de todas as operações com o LCD e o teclado.

4.6 M

A classe **M** (descrita em B.2) é responsável por gerir o comportamento do sistema quando este se encontra em modo de manutenção. Este modo é ativado automaticamente sempre que o bit associado ao switch de manutenção (**máscara 0x80**) se encontra ativo. Durante este modo, o sistema interrompe o jogo normal e apresenta um menu de opções de manutenção no LCD.

Ao entrar em manutenção (**enterMaintenance()**), o sistema alterna ciclicamente entre duas mensagens informativas no LCD: "*-Play D-ShutD"e "C-Stats A-Count", para orientar o utilizador sobre as ações disponíveis. A cada 5 segundos, verifica se foi premida alguma tecla com significado especial:
D → Desligar o sistema, com confirmação por tecla 5. Neste caso, os dados estatísticos são persistidos em ficheiro e o processo termina.
A → Consultar contadores (número de jogos realizados e moedas guardadas), com possibilidade de os reiniciar (*).
C → Consultar estatísticas da roleta, com navegação (2 - para cima e 8 - para baixo) e opção de apagar (* seguido de 5).
***** → Iniciar um jogo de teste, sem afetar os dados reais do sistema.

Jogo de Teste
Ao premir *****, o sistema entra num ciclo de jogo simulado com um saldo inicial de 100 créditos. O utilizador pode apostar nos símbolos válidos (0-9 e A-D), visualizando no LCD tanto o saldo como a quantidade de apostas feitas em cada símbolo. A aposta custa 1 crédito e é limitada a 9 por símbolo.

Ao premir #, inicia-se o sorteio. É então ativada uma animação de desaceleração no RouletteDisplay, seguida da exibição do símbolo sorteado, que pisca várias vezes. Dependendo do resultado, é apresentado no visor:
- o símbolo vencedor no primeiro dígito;
- os créditos ganhos (ou perdidos, com símbolo –) nos restantes.
Toda a lógica de animação, sorteio e apresentação é gerida por funções como **updateTestAnim()** e **showResultOnDisplay()**.

Gestão de Dados
O módulo interage com os ficheiros "statistics.txt"e "Roulette_Stats.txt"através do módulo FileAccess, garantindo que alterações feitas em manutenção (como apagar contadores ou estatísticas) são persistidas corretamente.

Função main
No main(), o sistema inicializa os módulos principais (HAL, KBD, LCD, SerialEmitter, RouletteDisplay) e carrega os ficheiros de estatísticas. De seguida, entra no modo de manutenção, que se mantém ativo enquanto o switch M estiver ligado.

4.7 APP

A classe **APP** (descrita em B.1) representa o núcleo do sistema, que é responsável por coordenar o ciclo do jogo da roleta. Este módulo integra todas as funcionalidades principais: controlo do saldo, receção de moedas, registo de apostas, sorteio do número vencedor, apresentação dos resultados no RouletteDisplay, e atualização dos ficheiros de estatísticas.

O ciclo principal é iniciado pela função **startGame()**, que executa continuamente cinco subfunções (appA a appE) de forma sequencial, garantindo uma divisão clara das responsabilidades:
appA(): Interface Inicial Apresenta o ecrã de boas-vindas ("Roulette Game"), com os caracteres especiais à esquerda e o saldo do jogador alinhado à direita da segunda linha do LCD. Este ecrã só é atualizado quando necessário.

appB(): Animação Se o utilizador ainda não estiver a apostar, é ativada a animação contínua do RouletteDisplay. Quando um sorteio está em curso, esta função invoca **updateDrawAnimation()** para mostrar uma sequência de símbolos animados, intercalados com o saldo atual.

appC(): Inserção de Moedas Deteta a inserção de moedas através do módulo CoinDeposit. Cada moeda válida incrementa o saldo e é registada em ficheiro. Esta operação é desativada durante o período de apostas e de sorteio.

appD(): Entrada no Modo de Manutenção Se o sistema detetar o bit de manutenção ativo e o jogo não estiver a decorrer, transfere o controlo para a classe M, passando as referências às estatísticas de jogo e da roleta. Após sair da manutenção, o sistema volta ao estado inicial.

appE(): Gestão das Teclas Esta função é responsável por interpretar as teclas introduzidas pelo utilizador. O jogo começa ao premir *, desde que o saldo seja positivo. O utilizador pode então apostar em qualquer um dos símbolos 0–9 e A–D, sendo cada aposta registada no map bets, com custo de 1 crédito.

Quando o jogador termina as apostas, premindo #, inicia-se o sorteio. É ativada uma animação de desaceleração no RouletteDisplay com símbolos em sequência, terminando no símbolo vencedor. Este símbolo pisca 10 vezes e, de seguida, é apresentado o prémio ganho (ou as perdas, com símbolo –), no Roulette Display.
O cálculo do prémio é feito com base no número de apostas no símbolo sorteado, atribuindo 2 créditos por cada aposta certa. O saldo é atualizado e todos os dados relevantes são guardados em ficheiro (CoinDeposit e StatisticsManager).

Outras funcionalidades
prepareForBetting() limpa o ecrã e escreve os símbolos válidos na linha inferior.
updateLCD() mostra, na linha superior, o número de apostas feitas por símbolo.
resetState() limpa as apostas, aceita novas moedas durante um pequeno intervalo, e reinicia o ecrã.

Inicialização no main()
Na main, são inicializados todos os módulos físicos e lógicos: HAL, KBD, LCD, SerialEmitter, RouletteDisplay, CoinDeposit, StatisticsManager e CoinAcceptor. Por fim, o jogo é iniciado com **APP.init()** e **APP.startGame()**.

5 Interligações entre Hardware e Software

A ligação entre o hardware físico e o software do sistema (módulo Control em Kotlin) é feita através do módulo UsbPort, que funciona como ponte de comunicação entre os dois domínios. Esta comunicação é orientada a bits e controlada diretamente pelas leituras e escritas realizadas pelo software nos registos do UsbPort.

A Figura 7 (representada abaixo) ilustra o mapeamento dos sinais utilizados nesta comunicação.

OUTPUT:

ACK	accept			RDsel	LCDsel	SCLK	SDX

INPUT:

M	Coin	Coinid	Dval	D	D	D	D

Figura 7: Sinais de entrada e saída do *UsbPort*

No ficheiro de simulação rouletteGame.simul, cada ligação entre o UsbPort e os módulos internos está descrita explicitamente:

Teclado (KeyboardReader):
O código da tecla premida é enviado pelo barramento D[3..0] e validado com Dval. O software responde com ACK para confirmar a leitura.

Moedeiro (CoinAcceptor):

O hardware sinaliza a deteção de uma moeda (Coin) e o tipo da moeda (Coinid). Após processar, o PC ativa o sinal accept via UsbPort.O6.

Modo de manutenção (Switch M):
A posição da chave física é lida diretamente no bit M (UsbPort.I7).

O software controla os periféricos visuais através de sinais série que seguem protocolos específicos:

LCD (SLCDC):
Controlado por sinais LCDsel, SCLK, SDX (outputs O2, O1, O0).
Os dados recebidos pelo SerialReceiverParity(5) são convertidos pelo LCD_Dispatcher nos sinais paralelos RS, E, e DLCD[7..4].

Roulette Display (SRC):
Utiliza sinais RDsel, SCLK, SDX (outputs O3, O1, O0).
O SerialReceiverParity(8) envia os dados para o Roulette_Dispatcher, que escreve nos registos de comando e dados do mostrador.

A arquitetura garante que a receção de dados pelos periféricos só ocorre quando o software ativa explicitamente os sinais LCDsel ou RDsel. A temporização é gerida pelo software através do controlo da linha de relógio (SCLK). O uso de registos (Register1) na entidade de topo assegura a estabilidade destes sinais síncronos.

Este ficheiro permite uma comunicação fiável, expansível e testável por simulação, sendo possível ativar e observar o comportamento individual de cada subcomponente (ex.: teclado, LCD, moedeiro) conforme ilustrado no ficheiro de simulação.

6 Resultados Experimentais

A verificação funcional do sistema foi realizada em duas fases: testes individuais aos módulos (hardware e software) e teste global ao jogo completo.

Testes Individuais
No hardware, foram desenvolvidos e executados tbs para validar o comportamento dos principais componentes descritos em VHDL. Estes testes permitiram confirmar a resposta correta aos sinais de entrada e a geração apropriada dos sinais de saída, incluindo a receção de teclas, controlo do LCD, e apresentação no mostrador da roleta.
No software, cada módulo foi testado de forma isolada através de funções main() específicas, como se observa nos ficheiros CoinAcceptor, TUI e M, etc. Estes testes permitiram validar funcionalidades como a deteção de moedas, a escrita no LCD, o modo de manutenção e a interação com o teclado. Por exemplo, no módulo TUI, foi possível testar diretamente o comportamento do LCD em resposta às teclas premidas.

Teste do Sistema Completo
A integração total do sistema foi testada através do ficheiro APP.kt, onde o jogo completo foi executado com todos os módulos conectados. Este teste foi realizado inicialmente com o simulador, recorrendo ao ficheiro rouletteGame.simul, e em aula, usando a placa DE10-lite com o sistema implementado em VHDL e ligado ao software Kotlin.

Durante a execução, foram testados os seguintes casos:
- Inserção de moedas e aumento de saldo (confirmado no LCD).
- Realização de apostas válidas com desconto do saldo.
- Sorteio com animação no RouletteDisplay.
- Apresentação do símbolo sorteado e dos créditos ganhos ou perdidos.
- Entrada no modo de manutenção e verificação das estatísticas e contadores.

Comportamento Observado
Em todos os testes, tanto em simulação como na placa real, o sistema apresentou o comportamento esperado. Todos os módulos funcionaram corretamente e comunicaram entre si sem erros. Os dados foram corretamente atualizados e guardados nos ficheiros statistics.txt e Roulette_Stats.txt.

7 Conclusões

O trabalho consistiu na implementação de um sistema completo do jogo da Roleta, integrando módulos de hardware (em VHDL) e software (em Kotlin). O objetivo principal foi permitir que um jogador realizasse apostas num conjunto de símbolos (0–9 e A–D), utilizando um teclado, visualizando o saldo e os resultados num visor LCD e num display, e podendo inserir créditos através de um moedeiro simulado.

O sistema foi desenvolvido de forma modular, com diferentes componentes interligados:
No **hardware**, foram implementados os módulos KeyboardReader, SLCDC, SRC e rouletteDisplay, todos ligados na entidade de topo Roulette_Game.
No **software**, foram desenvolvidos diversos ficheiros Kotlin para tratar da interface com os dispositivos (CoinAcceptor, TUI, StatisticsManager, etc.), bem como a lógica central do jogo (APP.kt) e o modo de manutenção (M.kt).

O sistema foi testado tanto em simulação, utilizando o ficheiro rouletteGame.simul, como fisicamente em laboratório, recorrendo à placa DE10-lite fornecida na aula. Os testes abrangeram funcionalidades individuais (como o LCD, moedeiro, teclado e display) e o funcionamento global da aplicação.

Os resultados experimentais confirmam que o circuito se comporta de acordo com os requisitos propostos. O sistema responde corretamente à introdução de moedas, à deteção de teclas, executa o sorteio com animação, apresenta os prémios corretamente e permite a consulta e reposição das estatísticas no modo de manutenção.

Este projeto permitiu aplicar e consolidar conhecimentos sobre sistemas embebidos, comunicação série, interface hardware-software e o desenvolvimento de sistemas digitais completos.

8 Trabalho Futuro

Apesar de o sistema implementado cumprir todos os objetivos funcionais, existem algumas decisões de projeto que poderão ser revistas e melhoradas em trabalhos futuros.

A primeira dessas melhorias prende-se com a comunicação com o LCD. Conforme indicado na arquitetura lógica apresentada na Figura 6, a comunicação com o LCD deveria ser realizada através do módulo SerialEmitter, que assegura o envio estruturado de dados série. No entanto, na nossa implementação em Kotlin, a escrita no LCD foi feita diretamente através da camada HAL, o que viola parcialmente a abstração proposta. Uma futura versão do projeto poderá adaptar o código de controlo do LCD para passar corretamente pelo SerialEmitter, tal como já acontece com o RouletteDisplay.

Outro ponto de melhoria diz respeito à implementação do módulo Key Scan. O nosso projeto utilizou a versão I da arquitetura sugerida, por ser a mais simples. Contudo, o enunciado apresenta também as versões II e III, que incluem melhorias como controlo de varrimento e registo de sinais de pressão. A adoção de uma dessas versões poderia trazer maior robustez e melhor deteção de teclas em contextos de maior carga ou interferência.

Por fim, a verificação de paridade foi realizada com base num contador síncrono (counterPC), como apresentado no nosso módulo ParityCheck. Embora funcional, esta abordagem foi referida pelo professor como sendo mais complexa do que o necessário. Em alternativa, a paridade poderia ser verificada usando uma simples cadeia de XORs acumulados, reduzindo assim a complexidade do circuito e a sua área em hardware.

Estas observações refletem oportunidades de otimização e refinamento do projeto, quer ao nível da aderência à arquitetura recomendada, quer na simplificação e eficiência dos circuitos implementados.

Referências

[1] Pilar Gonçalves, Margarita Sysoeva *KeyBoardReader datasheet*, 2025, ISEL.

[2] Pilar Gonçalves, Margarita Sysoeva *Serial LCD Controller datasheet*, 2025, ISEL.

[3] Pilar Gonçalves, Margarita Sysoeva *Serial Roulette Display Controller datasheet*, 2025, ISEL.

[4] Terasic *DE10-lite User Manual*, 2020, Terasic.

[5] Terasic *de10-Lite Exp. Board - Reference Manual*, 2022, Pedro Miguens Matutino

A Roulette Game - Hardware

A.1 Roulette Game.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Roulette_Game is
        port(
        RESET :  in std_logic;
        CLK : in std_logic;
        KeyPadL : in std_logic_vector (3 downto 0);
        Coin : in std_logic;
        Coinid : in std_logic;
        M : in std_logic;
        KeyPadC : out std_logic_vector (3 downto 0);
        Dval : out std_logic;
        D : out std_logic_vector (3 downto 0);
        RS : out std_logic;
        E : out std_logic;
        DLCD : out std_logic_vector (7 downto 4);
        HEX0        : out std_logic_vector(7 downto 0);
        HEX1        : out std_logic_vector(7 downto 0);
        HEX2        : out std_logic_vector(7 downto 0);
        HEX3        : out std_logic_vector(7 downto 0);
        HEX4        : out std_logic_vector(7 downto 0);
        HEX5        : out std_logic_vector(7 downto 0);
        accept : out std_logic
        );
end Roulette_Game;

architecture structural of Roulette_Game is

component KeyboardReader is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        Dval : out std_logic;
        D : out std_logic_vector (3 downto 0);
        KeyPadL : in std_logic_vector (3 downto 0);
        KeyPadC : out std_logic_vector (3 downto 0);
        ACK : in std_logic
        );
end component;

component SLCDC is
        port(
        SS: in std_logic;
        SDX: in std_logic;
        SCLK:in std_logic;
        MCLK: in std_logic;
        RESET: in std_logic;
        WrL:out std_logic;
        Dout: out std_logic_vector(4 downto 0)
```

```
        );
end component;

component rouletteDisplay is
port(        set        : in std_logic;
                cmd        : in std_logic_vector(2 downto 0);
                data        : in std_logic_vector(4 downto 0);
                HEX0        : out std_logic_vector(7 downto 0);
                HEX1        : out std_logic_vector(7 downto 0);
                HEX2        : out std_logic_vector(7 downto 0);
                HEX3        : out std_logic_vector(7 downto 0);
                HEX4        : out std_logic_vector(7 downto 0);
                HEX5        : out std_logic_vector(7 downto 0)
                );
end component;

component SRC is
        port(
        SS: in std_logic;
        SDX: in std_logic;
        SCLK:in std_logic;
        MCLK: in std_logic;
        RESET: in std_logic;
        WrD:out std_logic;
        Dout: out std_logic_vector(7 downto 0)
        );
end component;

component UsbPort is
        port(
                inputPort:  IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
                outputPort :  OUT  STD_LOGIC_VECTOR(7 DOWNTO 0)
        );
end component;

component Register1 is
port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Dval_usb : std_logic;
signal D_usb : std_logic_vector(3 downto 0);
signal usb_ACK : std_logic;

signal DLCD_exit : std_logic_vector(7 downto 4);
signal E_exit : std_logic;
signal RS_exit : std_logic;

signal usb_SS_SRC_reg : std_logic;
```

```
signal usb_SS_reg : std_logic;
signal usb_SCLK_reg : std_logic;
signal usb_SDX_reg : std_logic;

signal reg_SS_SRC : std_logic;
signal reg_SS : std_logic;
signal reg_SCLK : std_logic;
signal reg_SDX : std_logic;

signal WrD_set : std_logic;
signal D_cmd : std_logic_vector(2 downto 0);
signal D_data: std_logic_vector(4 downto 0);

begin
U0: KeyboardReader port map (
                        CLK => CLK,
                        RESET => RESET,
                        Dval => Dval_usb,
                        D(0) => D_usb(0),
                        D(1) => D_usb(1),
                        D(2) => D_usb(2),
                        D(3) => D_usb(3),
                        KeyPadL(0) => KeyPadL(0),
                        KeyPadL(1) => KeyPadL(1),
                        KeyPadL(2) => KeyPadL(2),
                        KeyPadL(3) => KeyPadL(3),
                        KeyPadC(0) => KeyPadC(0),
                        KeyPadC(1) => KeyPadC(1),
                        KeyPadC(2) => KeyPadC(2),
                        KeyPadC(3) => KeyPadC(3) ,
                        ACK => usb_ACK );

U1: UsbPort port map(
        inputPort(4) => Dval_usb,
        inputPort(0) => D_usb(0),
        inputPort(1) => D_usb(1),
        inputPort(2) => D_usb(2),
        inputPort(3) => D_usb(3),
        inputPort(5) => Coinid,
        inputPort(6) => Coin,
        inputPort(7) => M,
        outputPort(7) => usb_ACK,
        outputPort(6) => accept,
        outputPort(3) => usb_SS_SRC_reg,
        outputPort(2) => usb_SS_reg,
        outputPort(1) => usb_SCLK_reg,
        outputPort(0) => usb_SDX_reg);


U2: SLCDC port map(
                        SS => reg_SS,
                        SDX => reg_SDX,
                        SCLK => reg_SCLK,
                        MCLK => CLK,
```

```
                        RESET => RESET,
                        WrL => E_exit,
                        Dout(0) => RS_exit,
                        Dout(1) => DLCD_exit(4),
                        Dout(2) => DLCD_exit(5),
                        Dout(3) => DLCD_exit(6),
                        Dout(4) => DLCD_exit(7));

U3: rouletteDisplay port map(
                        set => WrD_set,
                        cmd => D_cmd,
                        data => D_data,
                        HEX0 => HEX0,
                        HEX1 => HEX1,
                        HEX2 => HEX2,
                        HEX3 => HEX3,
                        HEX4 => HEX4,
                        HEX5 => HEX5);

U4: SRC port map(
                    SS => reg_SS_SRC,
                        SDX => reg_SDX,
                        SCLK => reg_SCLK,
                        MCLK => CLK,
                        RESET => RESET,
                        WrD => WrD_set,
                        Dout(0) => D_cmd(0),
                        Dout(1) => D_cmd(1),
                        Dout(2) => D_cmd(2),
                        Dout(3) => D_data(0),
                        Dout(4) => D_data(1),
                        Dout(5) => D_data(2),
                        Dout(6) => D_data(3),
                        Dout(7) => D_data(4) );

U5: Register1 port map(
                        CLK => CLK,
                        RESET => RESET,
                        EN => '1',
                        D(0) => usb_SDX_reg,
                        D(1) => usb_SCLK_reg,
                        D(2) => usb_SS_reg,
                        D(3) => usb_SS_SRC_reg,
                        Q(0) => reg_SDX,
                        Q(1) => reg_SCLK,
                        Q(2) => reg_SS,
                        Q(3) => reg_SS_SRC );

D <= D_usb;
Dval <= Dval_usb;

RS <= RS_exit;
E <= E_exit;
DLCD <= DLCD_exit;
```

```
end structural;
```

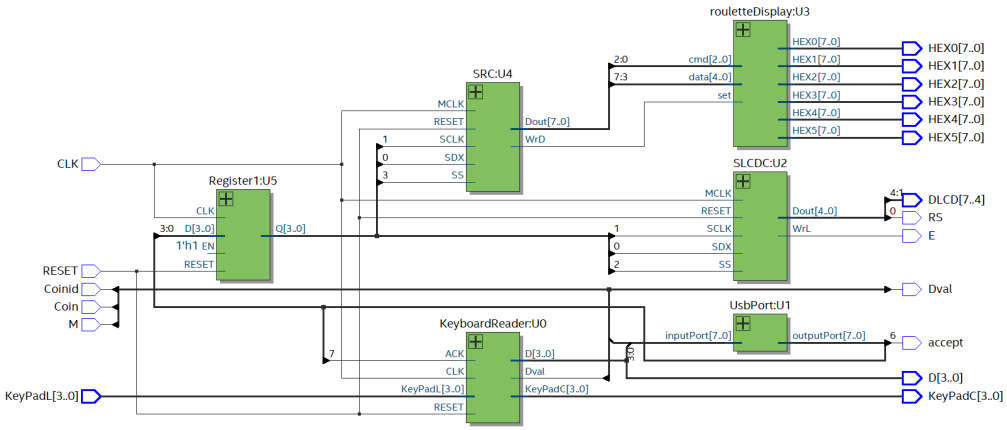


Figura 8: Netlist Viewer do *Roulette Game*

A.1.1 Register1.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Register1 is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end Register1;

architecture structural of Register1 is

component FFD2 is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
        );
end component;

begin

U0: FFD2 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD2 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
```

```
U2: FFD2 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD2 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.2 Atribuição de Pinos

Tabela 1: Atribuição de pinos para o Roulette Game [4]

Entradas	**pino**
RESET	F15
CLK	P11
KeyPadL[0]	W5
KeyPadL[1]	AA14
KeyPadL[2]	W12
KeyPadL[3]	AB12
M	C10
Coin	B14
Coinid	A14
Saídas	**pino**
accept	A11
KeyPadC[0]	AB11
KeyPadC[1]	AB10
KeyPadC[2]	AA9
KeyPadC[3]	AA8
D[0]	A8
D[1]	A9
D[2]	A10
D[3]	B10
Dval	D13
DLCD[4]	W11
DLCD[5]	AA10
DLCD[6]	Y8
DLCD[7]	Y7
RS	W8
E	V5
HEX0[0]	C14

Saídas	**pino**
HEX0[1]	E15
HEX0[2]	C15
HEX0[3]	C16
HEX0[4]	E16
HEX0[5]	D17
HEX0[6]	C17
HEX0[7]	D15
HEX1[0]	C18
HEX1[1]	D18
HEX1[2]	E18
HEX1[3]	B16
HEX1[4]	A17
HEX1[5]	A18
HEX1[6]	B17
HEX1[7]	A16
HEX2[0]	B20
HEX2[1]	A20
HEX2[2]	B19
HEX2[3]	A21
HEX2[4]	B21
HEX2[5]	C22
HEX2[6]	B22
HEX2[7]	A19
HEX3[0]	F21
HEX3[1]	E22
HEX3[2]	E21
HEX3[3]	C19
HEX3[4]	C20
HEX3[5]	D19
HEX3[6]	E17
HEX3[7]	D22
HEX4[0]	F18
HEX4[1]	E20
HEX4[2]	E19

Saídas	**pino**
HEX4[3]	J18
HEX4[4]	H19
HEX4[5]	F19
HEX4[6]	F20
HEX4[7]	F17
HEX5[0]	J20
HEX5[1]	K20
HEX5[2]	L18
HEX5[3]	N18
HEX5[4]	M20
HEX5[5]	N19
HEX5[6]	N20
HEX5[7]	L19

B Roulette Game - Software (*Control*)

B.1 Roulette Game - APP

Algoritmo 1: Roulette Game Application

```
package main.kotlin
import isel.leic.utils.Time


object APP {
    private var balance = 0
    private val bets = mutableMapOf<Char, Int>()
    private val validSymbols = "0123456789ABCD"
    private var isBetting = false
    private var isDrawing = false
    private var drawStartTime: Long = 0
    private var drawDuration: Long = 0
    private var drawAnimIndex = 0
    private var lastAnimUpdate: Long = 0
    private var needsScreenUpdate = true

    fun init() {
        balance = 0
        bets.clear()
        isBetting = false
        needsScreenUpdate = true
    }

    fun startGame() {
        while (true) {
            appA()  // Funcao de mostrar a tela inicial
            appB()  // Funcao de anima  o (n o-bloqueante)
            appC()  // Funcao de inserir moedas
            appD()  // Funcao de manuten  o
            appE()  // Funcao de processamento de teclas (* e #)
            Time.sleep(10)
        }
    }

    private fun appA() {
        if (!isBetting && needsScreenUpdate) {
            showStartScreen()
            needsScreenUpdate = false
        }
    }

    private fun appB() {
        if (!isBetting && !isDrawing) {
            RouletteDisplay.animation()
        } else if (isDrawing) {
            updateDrawAnimation()
        }
    }

    private fun appC() {
        if (!isBetting && !isDrawing) {
            val value = CoinDeposit.checkAndAccept()
            if (value > 0) {
                balance += value
                CoinDeposit.saveToFile()
                needsScreenUpdate = true
            }
```

```
        }
    }

    private fun appD() {
        if (M.onMaintenance() && !isBetting && !isDrawing) {
            val stats = CoinDeposit.getStatistics()
            val rouletteStats = StatisticsManager.getStats()
            if (stats != null && rouletteStats != null) {
                M.enterMaintenance(stats, rouletteStats)
                repeat(5) {
                    CoinDeposit.checkAndAccept()
                }
                needsScreenUpdate = true
            }
        }
    }

    private fun appE() {
        val key = TUI.waitKey(100)

        if (!isBetting && key == '*') {
            if (balance == 0) return
            RouletteDisplay.setNumber(balance)
            isBetting = true
            prepareForBetting()
        }

        if ((isBetting || isDrawing) && key in validSymbols && balance > 0) {
            processBet(key)
            updateLCD()
            RouletteDisplay.setNumber(balance)
        }

        if (key == '#' && isBetting && !isDrawing) {
            startDraw()
        }

        if (isDrawing && Time.getTimeInMillis() - drawStartTime >= drawDuration) {
            val drawn = validSymbols.random()
            slowDownAndStopOnWinner(drawn)
            val winnings = calculateWinnings(drawn)
            CoinDeposit.registerGame(winnings)
            CoinDeposit.saveToFile()
            StatisticsManager.saveToFile()
            resetState()
            isDrawing = false
        }
    }

    private fun showStartScreen() {
        TUI.clear()
        TUI.writeCentered(0, "Roulette␣Game")

        val specialChars = listOf(0, 1, 2, 3).map { it.toChar() }
        val specialPart = specialChars.joinToString("␣")
        val balanceStr = "\$${balance}"

        val originalStartPos = maxOf(0, (16 - specialPart.length - balanceStr.length - 1))
        val startPos = maxOf(0, originalStartPos - 1)

        TUI.write(1, 0, "␣".repeat(16))

        TUI.write(1, startPos, specialPart)
```

```
    TUI.write(1, 16 - balanceStr.length, balanceStr)
}

private fun prepareForBetting() {
    TUI.clear()
    TUI.writeCentered(1, validSymbols)
}

private fun processBet(key: Char) {
    val current = bets[key] ?: 0
    if (current < 9) {
        bets[key] = current + 1
        balance -= 1
    }
}

private fun updateLCD() {
    val line = validSymbols.map { ch ->
        val qty = bets[ch] ?: 0
        if (qty == 0) ' ' else qty.digitToChar()
    }.joinToString("")

    val spaces = "␣".repeat(14 - line.length)
    TUI.write(0, 1, line + spaces)
}

private fun slowDownAndStopOnWinner(winner: Char) {
    val symbols = validSymbols.toList()
    val winnerIdx = symbols.indexOf(winner)
    var delay = 200L
    val delayIncrement = 300L

    for (i in 0..winnerIdx) {
        val code = RouletteDisplay.mapCharToCode(symbols[i])
        RouletteDisplay.setValue(code)
        Time.sleep(delay)
        delay += delayIncrement
    }

    val winnerCode = RouletteDisplay.mapCharToCode(winner)

    repeat(10) {
        RouletteDisplay.setValue(winnerCode)
        Time.sleep(250)
        RouletteDisplay.clear()
        Time.sleep(250)
    }

    val hits = bets[winner] ?: 0
    val winnings = hits * 2

    val digits = IntArray(6) { 0x1F }
    digits[0] = RouletteDisplay.mapCharToCode(winner)

    if (winnings > 0) {
        val resultStr = winnings.toString()
        val start = 6 - resultStr.length
        for (i in resultStr.indices) {
            digits[start + i] = resultStr[i].digitToInt()
        }
    } else {
        val resultStr = (bets.values.sum()).toString()
        val start = 6 - resultStr.length - 1
        digits[start] = 0x10
```

```
            for (i in resultStr.indices) {
                digits[start + 1 + i] = resultStr[i].digitToInt()
            }
        }

        RouletteDisplay.setValues(digits)
        Time.sleep(3000)
    }

    private fun startDraw() {
        isDrawing = true
        drawStartTime = Time.getTimeInMillis()
        drawDuration = 5000L
        drawAnimIndex = 0
        lastAnimUpdate = 0
    }

    private fun updateDrawAnimation() {
        if (!isDrawing) return
        val currentTime = Time.getTimeInMillis()
        if (currentTime - drawStartTime >= drawDuration) return
        if (currentTime - lastAnimUpdate < 100) return

        val animSymbols = RouletteDisplay.positions
        val balanceStr = balance.toString()
        val balanceDigits = balanceStr.map { it.digitToInt() }

        val values = IntArray(6) { 0x1F }
        for (i in balanceDigits.indices) {
            values[5 - i] = balanceDigits[balanceDigits.size - 1 - i]
        }
        for (i in 0..< 6 - balanceDigits.size) {
            values[i] = animSymbols[drawAnimIndex % animSymbols.size]
        }

        RouletteDisplay.setValues(values)
        drawAnimIndex++
        lastAnimUpdate = currentTime
    }

    private fun calculateWinnings(drawn: Char): Int {
        val hits = bets[drawn] ?: 0
        val winnings = hits * 2
        balance += winnings

        StatisticsManager.registerOutcome(drawn, winnings)
        StatisticsManager.saveToFile()
        return winnings
    }

    private fun resetState() {
        repeat(5) {
            CoinDeposit.checkAndAccept()
        }
        bets.clear()
        isBetting = false
        needsScreenUpdate = true
        Time.sleep(1000)
    }
}

fun main() {
    HAL.init()
    KBD.init()
```

```
    LCD.init()
    LCD.specialChar()
    LCD.returnToDDRAM()

    SerialEmitter.init()
    RouletteDisplay.init()

    CoinDeposit.initFromFile()
    StatisticsManager.initFromFile()
    CoinAcceptor.init()

    APP.init()
    APP.startGame()
}
```

B.2 M

Algoritmo 2: Maintenance

```
package main.kotlin

import isel.leic.utils.Time
import kotlin.system.exitProcess

object M {
    private var statsRef: Statistics? = null
    private var rouletteStatsRef: MutableList<RouletteNumberStat>? = null
    private const val M_MASK = 0x80

    private var isTestDrawing = false
    private var testStartTime: Long = 0
    private var testDuration: Long = 0
    private var testAnimIndex = 0
    private var lastTestAnimUpdate: Long = 0

    fun enterMaintenance(stats: Statistics, rouletteStats: MutableList<RouletteNumberStat>) {
        statsRef = stats
        rouletteStatsRef = rouletteStats
        showMaintenanceSymbol()

        val messages = listOf("*-Play D-ShutD", "C-Stats A-Count")
        var toggle = true
        val switchInterval = 5000L

        while (true) {
            if (!onMaintenance()) return  // comentar esta linha, para testar M individualmente!!

            TUI.clear()
            TUI.writeCentered(0, "On Maintenance")
            TUI.write(1, 0, " ".repeat(16))
            TUI.writeCentered(1, if (toggle) messages[0] else messages[1])
            toggle = !toggle

            val start = Time.getTimeInMillis()
            while (Time.getTimeInMillis() - start < switchInterval) {
                val key = TUI.waitKey(200)
                when (key) {
                    'D' -> if (confirmShutdown()) return
                    'A' -> {
                        if (statsRef != null) showCounters(statsRef!!)
                        break
                    }
                    'C' -> {
                        if (rouletteStatsRef != null) showRouletteStats(rouletteStatsRef!!)
```

```
                    break
                }
                '*' -> {
                    startTestGame()
                    break
                }
                KBD.NONE.toChar() -> continue
                else -> break
            }
        }
    }
}

private fun confirmShutdown(): Boolean {
    TUI.clear()
    TUI.writeCentered(0, "Shutdown")
    TUI.writeCentered(1, "5-Yes␣Other-No")
    val key = TUI.waitKey(10000)

    return if (key == '5') {
        if (statsRef != null && rouletteStatsRef != null) {
            FileAccess.writeStatistics("statistics.txt", statsRef!!)
            FileAccess.writeRouletteStats("Roulette_Stats.txt", rouletteStatsRef!!)
        }
        TUI.clear()
        //TUI.showMessage(0, "Shutting down...")
        TUI.showLoading(0)
        TUI.clear()
        exitProcess(0)
    } else {
        TUI.clear()
        TUI.writeCentered(0, "On␣Maintenance")
        false
    }
}

private fun showCounters(stats: Statistics) {
    TUI.clear()
    TUI.write(0, 0, "Games:␣${stats.gamesPlayed}")
    TUI.write(1, 0, "Coins:␣${stats.coinsStored}")

    val key = TUI.waitKey(5000)
    if (key == '*') {
        TUI.clear()
        TUI.writeCentered(0, "Clear␣counters")
        TUI.writeCentered(1, "5-Yes␣␣␣other-No")

        val confirmKey = TUI.waitKey(5000)
        if (confirmKey == '5') {
            stats.gamesPlayed = 0
            stats.coinsStored = 0
            FileAccess.writeStatistics("statistics.txt", stats)
        }
    }
}

private fun showRouletteStats(stats: MutableList<RouletteNumberStat>) {
    var index = 0

    while (true) {
        TUI.clear()
        repeat(2) { line ->
            val statIndex = index + line
            if (statIndex < stats.size) {
```

```
                val stat = stats[statIndex]
                val symbol = when (stat.number) {
                    in 0..9 -> stat.number.toString()
                    10 -> "A"
                    11 -> "B"
                    12 -> "C"
                    13 -> "D"
                    else -> stat.number.toString()
                }
                TUI.write(line, 0, "$symbol: -> ${stat.count} \$:${stat.prizes}")
            }
        }

        val key = TUI.waitKey(5000)

        when (key) {
            '8' -> if (index + 2 < stats.size) index += 1
            '2' -> if (index > 0) index -= 1
            '*' -> {
                TUI.clear()
                TUI.writeCentered(0, "Clear statistics")
                TUI.writeCentered(1, "5-Yes  other-No")
                val confirm = KBD.waitKey(5000)
                if (confirm == '5') {
                    stats.forEach {
                        it.count = 0
                        it.prizes = 0
                    }
                    FileAccess.writeRouletteStats("Roulette_Stats.txt", stats)
                }
                return
            }
            else -> return
        }
    }
}

private fun startTestGame() {
    var balance = 100
    val bets = mutableMapOf<Char, Int>()
    val validSymbols = "0123456789ABCD"
    var isBetting = true

    TUI.clear()
    TUI.writeCentered(1, validSymbols)
    RouletteDisplay.setNumber(balance)

    while (true) {
        if (isTestDrawing) {
            updateTestAnim(balance)
        }

        val key = TUI.waitKey(100)

        if ((isBetting || isTestDrawing) && key in validSymbols && balance > 0) {
            val current = bets[key] ?: 0
            if (current < 9) {
                bets[key] = current + 1
                balance -= 1

                // Atualiza a linha 0 com quantidade apostada
                val line = validSymbols.map { ch ->
                    val qty = bets[ch] ?: 0
                    if (qty == 0) ' ' else qty.digitToChar()
```

```
                }.joinToString("")
                TUI.write(0, 1, line + "␣".repeat(14 - line.length))

                // Atualiza o saldo
                RouletteDisplay.setNumber(balance)
            }
        }

        if (key == '#' && isBetting && !isTestDrawing) {
            isTestDrawing = true
            testStartTime = Time.getTimeInMillis()
            testDuration = 5000L
            testAnimIndex = 0
            lastTestAnimUpdate = 0
        }

        if (isTestDrawing && Time.getTimeInMillis() - testStartTime >= testDuration) {
            val drawn = validSymbols.random()
            showResultOnDisplay(drawn, bets)
            Time.sleep(3000)
            isTestDrawing = false
            break
        }
        Time.sleep(10)
    }
    TUI.clear()
    RouletteDisplay.clear()
    showMaintenanceSymbol()
}

private fun showMaintenanceSymbol() = RouletteDisplay.setValue(0x17)

private fun showResultOnDisplay(winner: Char, bets: Map<Char, Int>) {
    val validSymbols = "0123456789ABCD"
    val winnerIdx = validSymbols.indexOf(winner)
    var delay = 200L
    val delayIncrement = 300L

    for (i in 0..winnerIdx) {
        val code = RouletteDisplay.mapCharToCode(validSymbols[i])
        RouletteDisplay.setValue(code)
        Time.sleep(delay)
        delay += delayIncrement
    }

    val winnerCode = RouletteDisplay.mapCharToCode(winner)

    repeat(10) {
        RouletteDisplay.setValue(winnerCode)
        Time.sleep(250)
        RouletteDisplay.clear()
        Time.sleep(250)
    }

    val hits = bets[winner] ?: 0
    val winnings = hits * 2

    val digits = IntArray(6) { 0x1F }
    digits[0] = RouletteDisplay.mapCharToCode(winner)

    if (winnings > 0) {
        val resultStr = winnings.toString()
        val start = 6 - resultStr.length
        for (i in resultStr.indices) {
```

```
                digits[start + i] = resultStr[i].digitToInt()
            }
        } else {
            // Se perdeu cr ditos , mostra com s mbolo -    esquerda
            val resultStr = (bets.values.sum()).toString()  // valor perdido foi o total apostado
            val start = 6 - resultStr.length - 1  // -1 para o s mbolo -
            digits[start] = 0x10 // s mbolo de menos
            for (i in resultStr.indices) {
                digits[start + 1 + i] = resultStr[i].digitToInt()
            }
        }
        RouletteDisplay.setValues(digits)
    }

    private fun updateTestAnim(balance: Int) {
        if (!isTestDrawing) return
        val currentTime = Time.getTimeInMillis()
        if (currentTime - testStartTime >= testDuration) return
        if (currentTime - lastTestAnimUpdate < 100) return

        val animSymbols = RouletteDisplay.positions
        val balanceStr = balance.toString()
        val balanceDigits = balanceStr.map { it.digitToInt() }

        val values = IntArray(6) { 0x1F }
        for (i in balanceDigits.indices) {
            values[5 - i] = balanceDigits[balanceDigits.size - 1 - i]
        }
        for (i in 0..< 6 - balanceDigits.size) {
            values[i] = animSymbols[testAnimIndex % animSymbols.size]
        }

        RouletteDisplay.setValues(values)
        testAnimIndex++
        lastTestAnimUpdate = currentTime
    }

    fun onMaintenance(): Boolean{
        return HAL.isBit(M_MASK)
    }
}

fun main() {
    HAL.init()
    KBD.init()
    LCD.init()
    SerialEmitter.init()
    RouletteDisplay.init()

    val stats = FileAccess.readStatistics("statistics.txt")
    val rouletteStats = FileAccess.readRouletteStats("Roulette_Stats.txt")
    M.enterMaintenance(stats, rouletteStats)
}
```

B.3 TUI

Algoritmo 3: TUI

```
import isel.leic.utils.Time

object TUI {

    fun tui() {
```

```
    LCD.clear() // Limpa o display
    LCD.cursor(0, 0) // Posiciona o cursor no inicio do display
    Time.sleep(50) // Pequena pausa para garantir a inicializa ao

    var column = 0 // Controla a posi ao da coluna
    var line = 0   // Controla a linha atual

    while (true) {
        val key = KBD.waitKey(5000) // Espera ate 5 segundos por uma tecla
        if (key != KBD.NONE.toChar()) { // Se uma tecla valida foi pressionada
            when (key) {
                '#' -> { // '#' limpa o display e reseta o cursor
                    LCD.clear()
                    Time.sleep(20) // Aguarda o LCD processar o comando de limpar
                    line = 0
                    column = 0
                    LCD.cursor(line, column) // Garante que o cursor volte ao inicio
                }
                '*' -> { // '*' muda para a proxima linha
                    if (line == 0) {
                        line = 1
                    } else {
                        line = 0
                    }
                    column = 0
                    LCD.cursor(line, column)
                }
                else -> {
                    LCD.write(key)
                    column++
                    if (column >= 16) { // Se atingir o fim da linha
                        column = 0
                        line++ // Avan a para a proxima linha
                        if (line >= 2) line = 0 // Volta para a primeira linha se atingir o limite (2
                        LCD.cursor(line, column) // Atualiza a posi ao do cursor
                    }
                }
            }
            println("Tecla pressionada: $key")
        }
    }
}

fun writeCentered(line: Int, text: String) {
    val startCol = maxOf((16 - text.length) / 2, 0)
    LCD.cursor(line, startCol)
    LCD.write(text)
}

fun showMessage(line: Int, text: String, durationMs: Long = 2000) {
    LCD.clear()
    writeCentered(line, text)
    Time.sleep(durationMs)
    LCD.clear()
}

fun showLoading(line: Int, ciclos: Int = 3, delayMs: Long = 400) {
    for (i in 1..ciclos * 3) {
        val pontos = (i - 1) % 3 + 1
        val texto = "Shutting down" + ".".repeat(pontos)
        LCD.cursor(line, 0)
        val final = texto + " ".repeat(16 - texto.length)
        LCD.write(final)
        Time.sleep(delayMs)
```

```
    }
    LCD.clear()
}

fun readInput(prompt: String, maxLen: Int = 10): String {
    LCD.clear()
    LCD.write(prompt)
    var input = ""
    var column = prompt.length
    LCD.cursor(1, 0)

    while (input.length < maxLen) {
        val key = KBD.waitKey(10000)
        if (key == '#') break
        if (key != KBD.NONE.toChar()) {
            input += key
            LCD.write(key)
            column++
        }
    }
    return input
}

fun scrollText(line: Int, text: String, durationMs: Long = 5000, delayMs: Long = 300) {
    if (text.length <= 16) {
        LCD.cursor(line, 0)
        LCD.write(text.padEnd(16, ' '))
        Time.sleep(durationMs)
        return
    }

    val start = Time.getTimeInMillis()
    while (Time.getTimeInMillis() - start < durationMs) {
        for (i in 0..(text.length - 16)) {
            if (Time.getTimeInMillis() - start >= durationMs) return
            LCD.cursor(line, 0)
            LCD.write(text.substring(i, i + 16))
            Time.sleep(delayMs)
        }
    }
}

fun blinkMessage(line: Int, text: String, vezes: Int = 4, intervalo: Long = 300) {
    val inicio = maxOf((16 - text.length) / 2, 0) // para centrar

    repeat(vezes) {
        LCD.cursor(line, inicio)
        LCD.write(text) // mostra o texto
        Time.sleep(intervalo)

        LCD.cursor(line, inicio)
        LCD.write("␣".repeat(text.length)) // apaga o texto
        Time.sleep(intervalo)
    }
}

fun write(line: Int, column: Int, text: String) {
    LCD.cursor(line, column)
    LCD.write(text)
}

fun clear() = LCD.clear()

fun waitKey(timeout: Long) = KBD.waitKey(timeout)
```

```
}

fun main() {
    HAL.init()
    KBD.init()
    SerialEmitter.init()
    LCD.init()
    //TUI.tui() // Testa escrever no LCD com o teclado


    while (true) {
        LCD.clear()
        TUI.writeCentered(0, "Escolha uma opcao")
        TUI.writeCentered(1, "* A B C D 0 #")

        val key = KBD.waitKey(10000)

        if (key != KBD.NONE.toChar()) {
            LCD.clear()  // limpa o LCD antes de qualquer a   o

            when (key) {
                'A' -> TUI.writeCentered(0, "Jogo Da Roleta")
                'B' -> TUI.showMessage(0, "Bem-vindo", 3000)
                'C' -> TUI.showLoading(0)
                'D' -> {
                    val id = TUI.readInput("Id: ")
                    TUI.showMessage(0, "Ola jogador num $id")
                }

                '*' -> TUI.scrollText(0, "Tenta a tua sorte no Jogo da Roleta!", 8500, 200)
                '0' -> TUI.blinkMessage(0, "Estas pronto?", 6, 250)
                '#' -> {
                    LCD.clear()
                    TUI.writeCentered(0, "Teste Terminado")
                    break
                }

                else -> TUI.writeCentered(1, "Tecla: $key")
            }

            Time.sleep(1500)
        }
    }
}
```

B.4 Coin Acceptor

Algoritmo 4: Coin Acceptor

```
package main.kotlin
import isel.leic.utils.Time

object CoinAcceptor {
    const val ACCEPT_MASK = 0x40
    const val COIN_MASK = 0x40
    const val COINID_MASK = 0x20
    private var lastCoinState: Boolean = false

    fun init () {
        HAL.clrBits(ACCEPT_MASK)
        lastCoinState = false
    }

    fun hasCoin () : Boolean {
```

```
        val currentCoinState = HAL.isBit(COIN_MASK)
        val coinDetected = !lastCoinState && currentCoinState
        lastCoinState = currentCoinState
        return coinDetected
    }

    fun getCoinValue (): Int {
        return if (HAL.isBit(COINID_MASK)) 4 else 2
    }

    fun acceptCoin () {
        HAL.setBits(ACCEPT_MASK)
        Time.sleep(200)
        HAL.clrBits(ACCEPT_MASK)
    }
}

fun main() {
    HAL.init()
    CoinAcceptor.init()

    while (true) {
        if (CoinAcceptor.hasCoin()) {
            CoinAcceptor.getCoinValue()
            CoinAcceptor.acceptCoin()
        }
        Time.sleep(100)
    }
}
```

B.5 Coin Deposit

Algoritmo 5: Coin Deposit

```
package main.kotlin

import isel.leic.utils.Time

object CoinDeposit {
    private var stats: Statistics? = null

    fun initFromFile(fileName: String = "statistics.txt") {
        stats = FileAccess.readStatistics(fileName)
    }

    fun getStatistics(): Statistics? = stats

    // Verifica se h   moeda, aceita-a e atualiza apenas coinsStored
    fun checkAndAccept(): Int {
        val currentStats = stats ?: return 0

        if (CoinAcceptor.hasCoin()) {
            val coinValue = CoinAcceptor.getCoinValue()
            CoinAcceptor.acceptCoin()
            currentStats.coinsStored += coinValue
            return coinValue
        }
        return 0
    }

    fun saveToFile(fileName: String = "statistics.txt") {
        if (stats != null) {
            FileAccess.writeStatistics(fileName, stats!!)
        }
```

```
    }

    fun registerGame(winnings: Int) {
        val currentStats = stats ?: return
        currentStats.gamesPlayed = (currentStats.gamesPlayed ?: 0) + 1
        currentStats.coinsStored -= winnings
    }
}

fun main() {
    HAL.init()
    CoinAcceptor.init()

    CoinDeposit.initFromFile()

    println("  espera de moedas...")
    while (true) {
        val value = CoinDeposit.checkAndAccept()
        if (value > 0) {
            println("Moeda aceite: $value cr ditos")
            println("Total moedas: ${CoinDeposit.getStatistics()?.coinsStored}")
            CoinDeposit.saveToFile()
        }
        Time.sleep(100)
    }
}
```

B.6 Statistics Manager

Algoritmo 6: Statistics Manager

```
package main.kotlin

object StatisticsManager {
    private var stats: MutableList<RouletteNumberStat>? = null

    fun initFromFile(fileName: String = "Roulette_Stats.txt") {
        stats = FileAccess.readRouletteStats(fileName)
    }

    fun getStats(): MutableList<RouletteNumberStat>? = stats

    private fun symbolToIndex(ch: Char): Int? = when (ch) {
        in '0'..'9' -> ch.digitToInt()
        'A' -> 10
        'B' -> 11
        'C' -> 12
        'D' -> 13
        else -> null
    }

    // Atualiza os dados de um n mero sorteado
    fun registerOutcome(number: Char, totalPrize: Int) {
        val statsList = stats ?: return
        val index = symbolToIndex(number) ?: return
        val entry = statsList.getOrNull(index) ?: return
        entry.count += 1
        entry.prizes += totalPrize
    }

    fun saveToFile(fileName: String = "Roulette_Stats.txt") {
        val currentStats = stats
        if (currentStats != null) {
            FileAccess.writeRouletteStats(fileName, currentStats)
```

```
        }
    }
}

fun main() {
    StatisticsManager.initFromFile()
    StatisticsManager.registerOutcome('A', 30)
    StatisticsManager.registerOutcome('3', 45)
    // Guarda os dados atualizados no ficheiro
    StatisticsManager.saveToFile()
    println("Estat sticas atualizadas com sucesso.")
}
```

B.7 File Access

Algoritmo 7: File Access

```
package main.kotlin

import java.io.File

data class RouletteNumberStat(val number: Int, var count: Int, var prizes: Int)

data class Statistics(var gamesPlayed: Int = 0, var coinsStored: Int = 0)

object FileAccess {
    private const val DELIM = ';'

    fun readStatistics(fileName: String): Statistics {
        val file = File(fileName)
        if (!file.exists() || file.readText().isBlank()) {
            file.writeText("0\n0\n")
            return Statistics()
        }
        val lines = file.readLines()
        val games = lines.getOrNull(0)?.toIntOrNull() ?: 0
        val coins = lines.getOrNull(1)?.toIntOrNull() ?: 0
        return Statistics(games, coins)
    }

    fun writeStatistics(fileName: String, stats: Statistics) {
        val file = File(fileName)
        file.writeText("${stats.gamesPlayed}\n${stats.coinsStored}\n")
    }

    fun readRouletteStats(fileName: String): MutableList<RouletteNumberStat> {
        val file = File(fileName)
        if (!file.exists()) {
            return (0..13).map { RouletteNumberStat(it, 0, 0) }.toMutableList()
        }
        val lines = file.readLines().filter { it.isNotBlank() }
        val stats = mutableListOf<RouletteNumberStat>()
        for (line in lines) {
            val parts = line.split(DELIM)
            if (parts.size == 3) {
                val number = parts[0].toIntOrNull() ?: continue
                val count = parts[1].toIntOrNull() ?: 0
                val prizes = parts[2].toIntOrNull() ?: 0
                stats.add(RouletteNumberStat(number, count, prizes))
            }
        }
        return stats
    }
```

```
    fun writeRouletteStats(fileName: String, stats: List<RouletteNumberStat>) {
        val file = File(fileName)
        val lines = stats.map { "${it.number}$DELIM${it.count}$DELIM${it.prizes}" }
        file.writeText(lines.joinToString("\n") + "\n")
    }
}

fun main() {
    val statsFile = "statistics.txt"
    val rouletteFile = "Roulette_Stats.txt"

    // Ler estat sticas
    val stats = FileAccess.readStatistics(statsFile)
    println("Jogos: ${stats.gamesPlayed}, Moedas: ${stats.coinsStored}")

    // Atualizar estat sticas e gravar
    stats.gamesPlayed += 1
    stats.coinsStored += 5
    FileAccess.writeStatistics(statsFile, stats)
    println("Estat sticas atualizadas e gravadas.")

    // Ler estat sticas da roleta
    val rouletteStats = FileAccess.readRouletteStats(rouletteFile)
    println("Estat sticas da Roleta:")
    rouletteStats.forEach {
        println("N mero ${it.number}: saiu ${it.count} vezes, pr mios: ${it.prizes}")
    }

    // Atualizar estat stica de um n mero e gravar
    if (rouletteStats.isNotEmpty()) {
        rouletteStats[0].count += 1
        rouletteStats[0].prizes += 10
        FileAccess.writeRouletteStats(rouletteFile, rouletteStats)
        println("Estat sticas da roleta atualizadas e gravadas.")
    }
}
```

C Roulette Game - Simulation

```
## --- Additional packages with Hardware to Simulator
# for Modules used in Roulette Game
package roulette.simul

# Generic modules to activate from Simulator
UsbPort = UsbPort
kbd  = Keyboard("123A456B789C*0#D",4,4,0)
lcd  = LCD
m    = Switch ; "manut" ; setLabel("M")

# Costume modules from roulette package
rb  = RingBuffer
ob  = OutputBuffer
srl = SerialReceiverParity(5)
dl  = LCD_Dispatcher
srr = SerialReceiverParity(8)
rdispatch = Roulette_Dispatcher
rdisplay = RouletteDisplay
ca = CoinAcceptor ; "Acceptor"

## --- Project Links ---
# --------------------------------------
# Manut
# --------------------------------------
m.out -> UsbPort.I7

# --------------------------------------
# Coin Acceptor
# --------------------------------------
UsbPort.O6 -> ca.accept
ca.Coin -> UsbPort.I6
ca.Coin_id -> UsbPort.I5

# --------------------------------------
# Keyboard Reader
# --------------------------------------
# Key Decode
1 -> kbd.oe
kbd.val -> rb.DAV
kbd.K[0-3] -> rb.D[0-3]

# Ring Buffer
rb.DAC -> kbd.ack
rb.Q[0-3] -> ob.D[0-3]
rb.Wreg -> ob.Load

# Output Buffer
ob.OBfree -> rb.CTS
ob.Q[0-3] -> UsbPort.I[0-3]
ob.Dval -> UsbPort.I4
UsbPort.O7 -> ob.ACK
```

```
# --------------------------------------
# SLCDC
# --------------------------------------
# Serial Receiver
UsbPort.O2 -> srl./SS
UsbPort.O0 -> srl.SDX
UsbPort.O1-> srl.SCLK
srl.DXval -> dl.Dval
srl.D[0-4] -> dl.I[0-4]

# LCD Dispatcher
dl.D[1-4] -> lcd.D[4-7]
dl.D0 -> lcd.rs
dl.WrL -> lcd.e
dl.done -> srl.accept

# --------------------------------------
# SRD
# --------------------------------------
# Serial Receiver
UsbPort.O3  -> srr./SS
UsbPort.O0  -> srr.SDX
UsbPort.O1  -> srr.SCLK
srr.Dxval   -> rdispatch.Dval
srr.D[0-7]  -> rdispatch.I[0-7]

# Roulette Dispatcher
rdispatch.done     -> srr.accept
rdispatch.WrD      -> rdisplay.set
rdispatch.D[0-2]   -> rdisplay.cmd[0-2]
rdispatch.D[3-7]   -> rdisplay.data[0-4]
```